

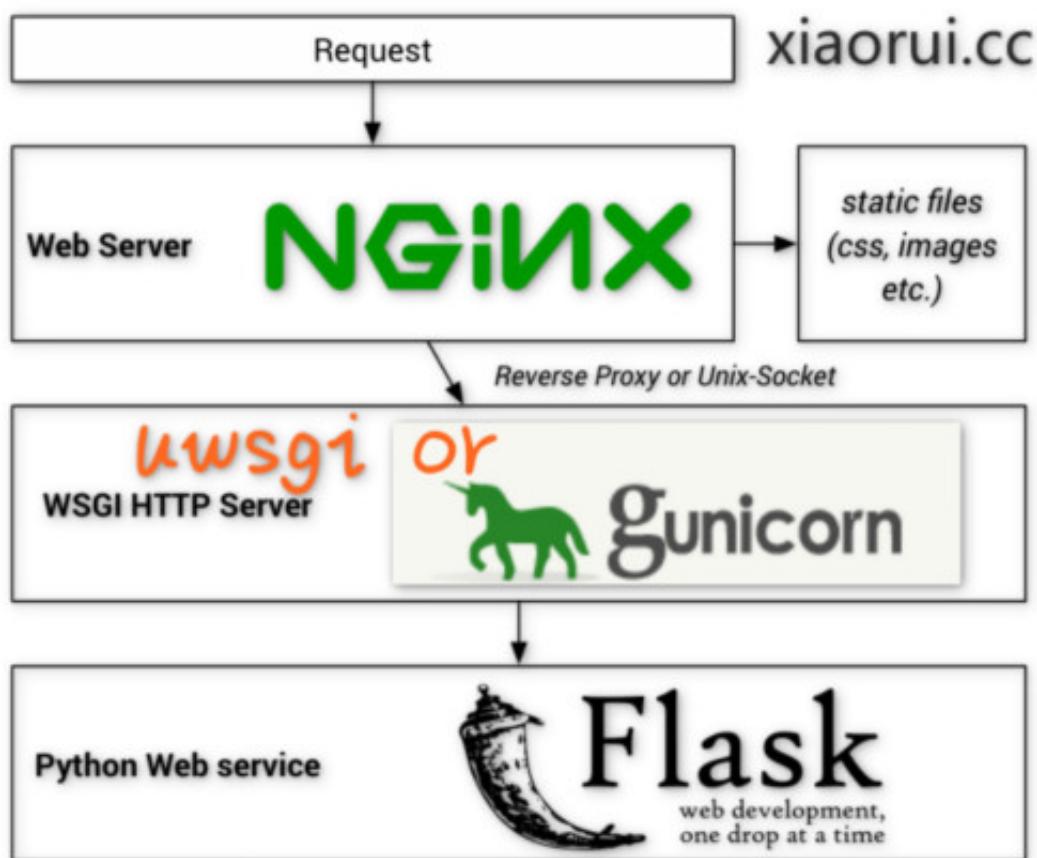
flask实践指南

- request-response
- flask 错误实践
- flask 基础知识
- flask 最佳实践
- RPC

一. REQUEST-RESPONSE

- WSGI
- uwsgi or gunicorn
- FLASK APP

1.WSGI



Web服务器网关接口（Python Web Server Gateway Interface，缩写为WSGI）是为Python语言定义的Web服务器和Web应用程序或框架之间的一种简单而通用的接口。自从WSGI被开发出来以后，许多其它语言中也出现了类似接口。

WSGI区分为两个部分：

- “服务器”或“网关”，
- “应用程序”或“应用框架”。在处理一个WSGI请求时，

服务器会为应用程序提供环境信息及一个回调函数（**Callback Function**）。当应用程序完成处理请求后，通过前述的回调函数，将结果回传给服务器。

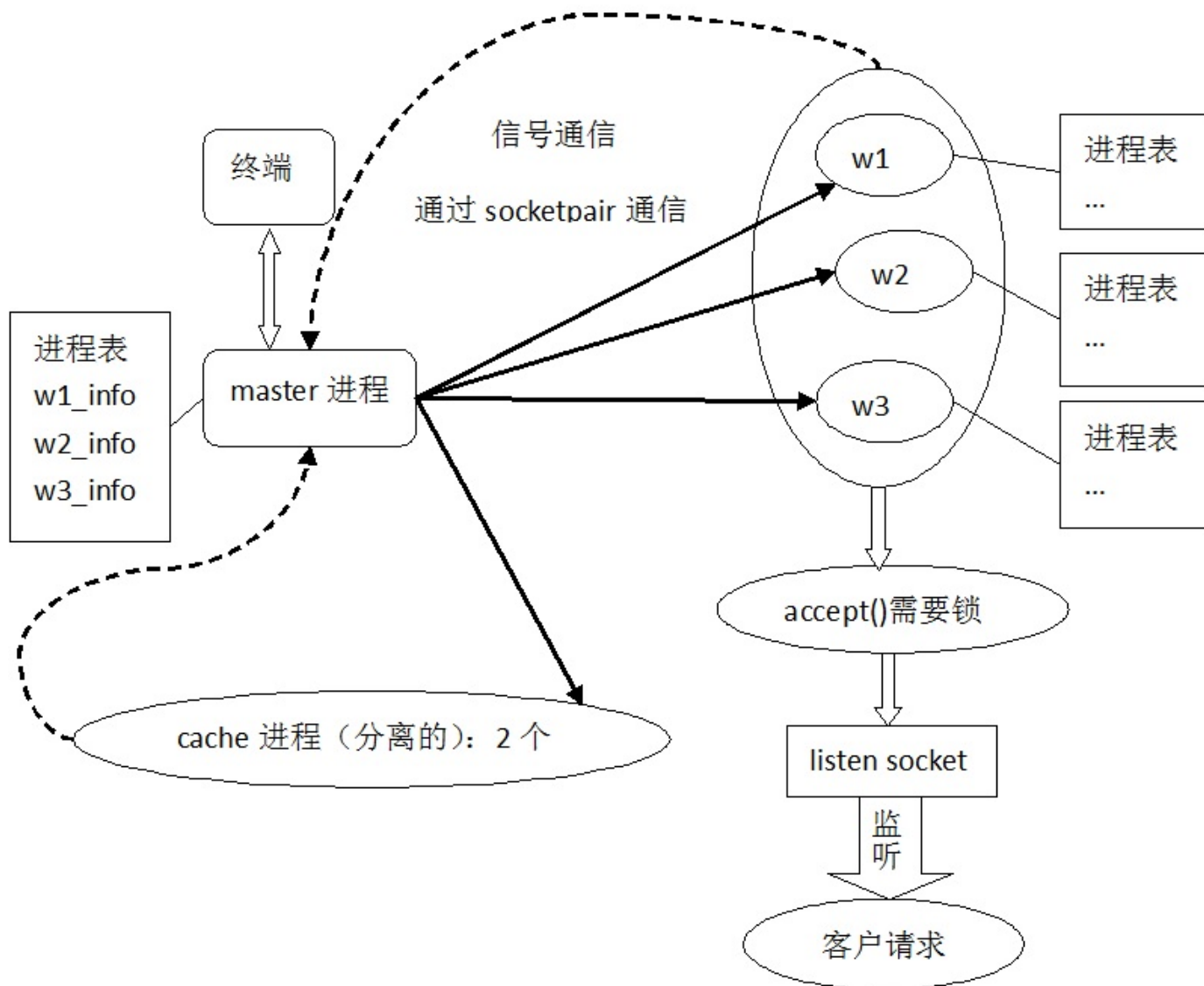
所谓的 WSGI 中间件同时实现了API的两方，因此可以在WSGI服务器和WSGI应用之间起调解作用：从WSGI服务器的角度来说，中间件扮演应用程序，而从应用程序的角度来说，中间件扮演服务器。

参考资料

- [WSGI: Web Server Gateway Interface](#)

2.uwsgi or gunicorn

gunicorn 、uwsgi 的 Master Worker的模型



werkzeug: 同步阻塞

gunicorn: 异步模型, 多进程, 多线程

- 当事件没准备好时, 放到epoll里面, 事件准备好了, 我们就去读写
- 并发请求, 是指未处理完的请求, 线程只有一个, 所以同时能处理的请求当然只有一个了, 只是在请求间进行不断地切换而已
- workflow之间会产生切换的, 但这里的切换消耗远没有多线程上下文切换大。

uwsgi

```

1  [uwsgi]
2  http=127.0.0.1:8888
3  wsgi-file=/Users/chenzikun/Documents/Test/soapa/run.py
4  callable=app
5  touch-reload=/Users/chenzikun/Documents/Test/soapa

```

gunicorn

```
#!/usr/bin/env bash  
gunicorn --workers=4 run:app -b 0.0.0.0:8080 -k gevent
```

参考资料

- [Master Worker的模型](#)
- [epoll模型](#)

3.FLASK APP

类作为回调对象

使用 `__call__` 对象上下文就被保留下来了

```

def wsgi_app(self, environ, start_response):
    """The actual WSGI application. This is not implemented in
    `__call__` so that middlewares can be applied without losing a
    reference to the class. So instead of doing this::

        app = MyMiddleware(app)

    It's a better idea to do this instead::

        app.wsgi_app = MyMiddleware(app.wsgi_app)

    Then you still have the original application object around and
    can continue to call methods on it.

    .. versionchanged:: 0.7
        The behavior of the before and after request callbacks was changed
        under error conditions and a new callback was added that will
        always execute at the end of the request, independent on if an
        error occurred or not. See :ref:`callbacks-and-errors`.

    :param environ: a WSGI environment
    :param start_response: a callable accepting a status code,
                           a list of headers and an optional
                           exception context to start the response
    """
    ctx = self.request_context(environ)
    ctx.push()
    error = None
    try:
        try:
            response = self.full_dispatch_request()
        except Exception as e:
            error = e
            response = self.handle_exception(e)
        except:
            error = sys.exc_info()[1]
            raise
        return response(environ, start_response)
    finally:
        if self.should_ignore_error(error):
            error = None
        ctx.auto_pop(error)

def __call__(self, environ, start_response):
    """Shortcut for :attr:`wsgi_app`."""
    return self.wsgi_app(environ, start_response)

```

二. flask 错误实践

1. 无工厂函数，不能创建不同环境的app，必须依赖uwsgi运行



```
28
29
30 # 初始化kafka连接池
31 init_kafka_connect_pool()
32 application = Flask(__name__)
33 application.config.from_object(os.environ.get(
34     'SOAPA_CLOUD_CONFIG_OBJECT', 'soapa_cloud.config.DebugConfig'))
35 init_component(application)
36 register_blueprints(application, 'soapa_cloud.blueprints', '/soapa')
37
```

2.配置文件放在model层， model层大部分不是model, 而是工具



```
22
23 def register_blueprints(app, package_name, url_prefix):
24     for name in find_modules(package_name):
25         bp_module = import_string(name)
26         if hasattr(bp_module, 'bp'):
27             app.register_blueprint(bp_module.bp, url_prefix=url_prefix)
28
29
30 # 初始化kafka连接池
31 init_kafka_connect_pool()
```

3.paramter线程不安全 另外它的作用是用来传递参数，应该直接使用request，如果的确是传递变量，应该使用flask.g 全局使用的值最好不要使用数组等魔力数字

```

3      import ...
12      reload(sys)
13      sys.setdefaultencoding('utf8')
14
15      bp = Blueprint('data_transfer', __name__)
16      log_list = ["None", "0.0.0.0", "None", "0.0"]
17      parameter = ([])
18
19      dns_log_moduleId = "ti_data_transfer"
20      logger = logging.getLogger(dns_log_moduleId)
21
22      def analytical_parameters(function_name):...
52
53
54      @bp.route('/data_transfer/request_unknown_data', methods=['POST'])
55      def request_unknown_data():
56          """
57          soapa平台对接云主机端请求数据接口
58          :return:
59          """
60          global parameter
61          analytical_parameters("request_unknown_data")
62          logger.info('#####')
63          try:

```

```

def get_data_from_kafka(parameters):
    """
    获取kafka数据
    :param parameters:
    :return:
    """
    try:
        source = parameters[0]
        types = parameters[4]
        host_topic_list = common_config.host_topic[types]
        source_topic_list = common_config.source_topic[source]

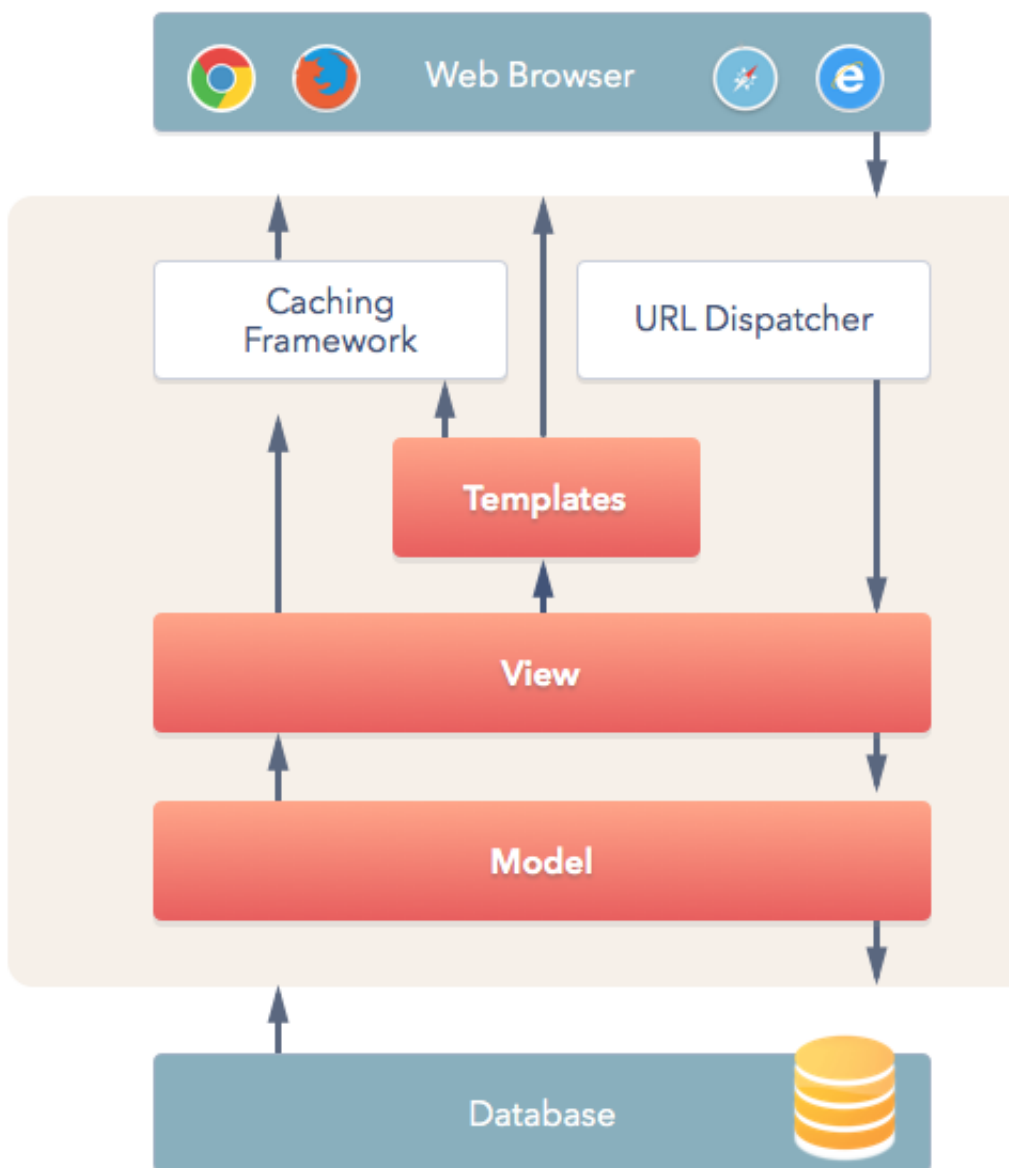
        # 求list交集
        topic = list(set(host_topic_list).intersection(set(source_topic_list)))

```

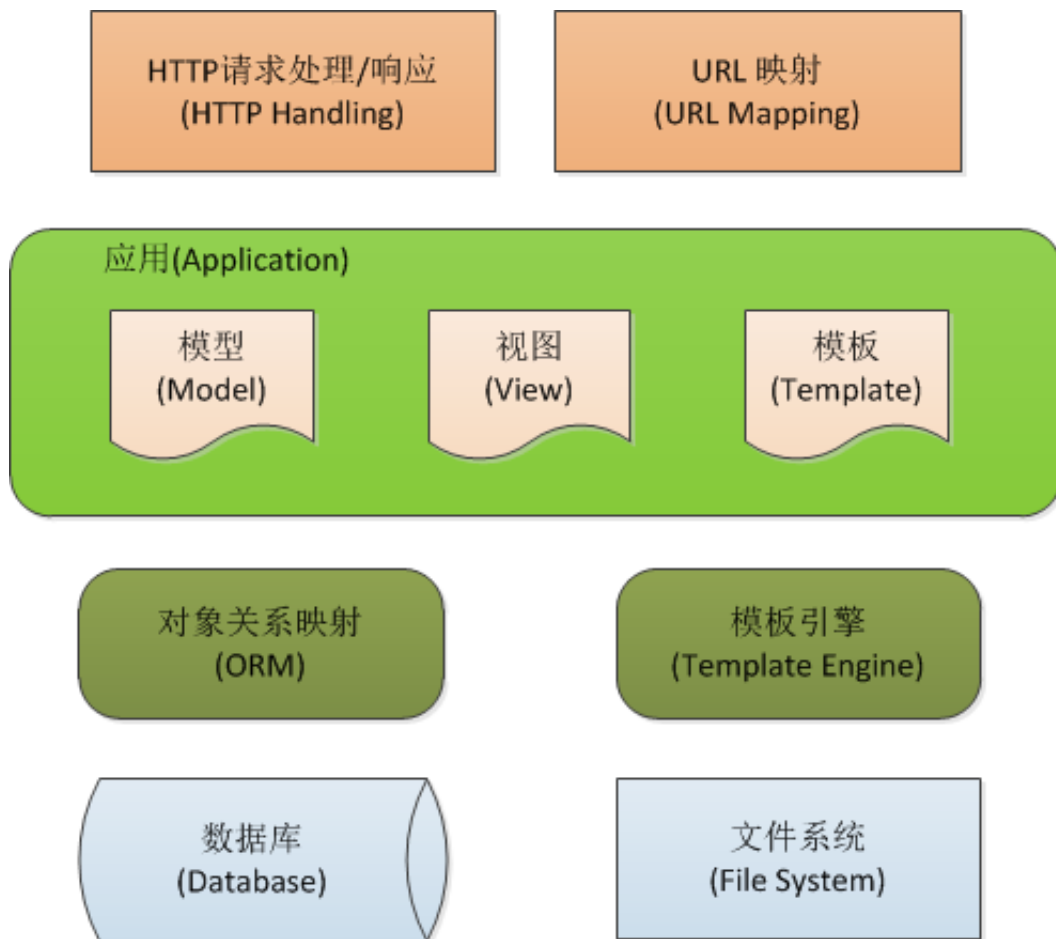
三. Flask基础知识

- MVC模型
- 应用上下文和请求上下文
- config
- extension
- 测试
- 模板templates
- 蓝图

1.MVC模型



django 架构图



Django架构总览图

2. 应用上下文和请求上下文

应用上下文模型和请求上下文模型

```
1 class AppContext:
2     def __enter__
3     def __exit__
4
5 class RequestContext:
6     def __enter__
7     def __exit__
8
9 with app.app_context().push() as current_app:
10     with app.request_context().push() as request:
11         handle_request()
12
13 class Stack:
14     def push
15     def pop
```

Python

Werkzeug 实现的 Local、Local Stack 和 Local Proxy

Werkzeug 没有直接使用 `threading.local`，而是自己实现了 `werkzeug.local.Local` 类。后者和前者有一些区别：

- Local 会在 Greenlet 可用的情况下优先使用 Greenlet 的 ID 而不是线程 ID 以支持 Gevent 或 Eventlet 的调度，前者只支持多线程调度；
- LocalStack 是用 Local 实现的栈结构，可以将对象推入、弹出，也可以快速拿到栈顶对象。当然，所有的修改都只在本线程可见。和 Local 一样，LocalStack 也同样实现了支持 `release_pool` 的接口。
- LocalProxy 则是一个典型的代理模式实现，它在构造时接受一个 callable 的参数（比如一个函数），这个参数被调用后的返回值本身应该是一个 Thread Local 对象。对一个 LocalProxy 对象的所有操作，包括属性访问、方法调用甚至是二元操作都会转发到那个 callable 参数返回的 Thread Local 对象上。

参考资料

- [Flask 的 Context 机制](#)

3. config

- config.py
- `class Config`
- config.json

```
class Config(dict):  
    """  
    ...  
    """  
  
    def __init__(self, root_path, defaults=None):  
        dict.__init__(self, defaults or {})  
        self.root_path = root_path  
  
    def from_envvar(self, variable_name, silent=False): ...  
  
    def from_pyfile(self, filename, silent=False): ...  
  
    def from_object(self, obj): ...  
  
    def from_json(self, filename, silent=False): ...
```

4. extension

委托模式

```

1 class FlaskExtension(object):
2     def __init__(self, app=None):
3         if app:
4             self.init_app(app)
5
6     def init_app(self, app):
7         if not hasattr(app, 'extensions'):
8             app.extensions = {}
9             app.extensions['EXTENSION'] = self

```

- 不直接赋值 `self.app = app`
- Extensions should be designed to work with multiple app
- Should always use `current_app` if the app object is needed ### 5. 测试

使用`app.test_client`构建请求上下文，不需手动启动和销毁程序

相当于直接测试flask，既没有用werkzeug的`run_simple`，也没有用uwsgi

```

1 class FlaskrTestCase(unittest.TestCase):
2
3     def setUp(self):
4         app = create_app()
5         self.db_fd, app.config['DATABASE'] = tempfile.mkstemp()
6         self.client = app.test_client()
7         flaskr.init_db()
8
9     def tearDown(self):
10        os.close(self.db_fd)
11        os.unlink(flaskr.DATABASE)
12
13    def test_empty_db(self):
14        rv = self.client.get('/')
15        assert 'No entries here so far' in rv.data

```

最新的flask文档已经推荐使用pytest去测试

```
1 import os
2 import tempfile
3
4 import pytest
5
6 from flaskr import flaskr
7
8
9 @pytest.fixture
10 def client():
11     db_fd, flaskr.app.config['DATABASE'] = tempfile.mkstemp()
12     flaskr.app.config['TESTING'] = True
13     client = flaskr.app.test_client()
14
15     with flaskr.app.app_context():
16         flaskr.init_db()
17
18     yield client
19
20     os.close(db_fd)
21     os.unlink(flaskr.app.config['DATABASE'])
```

6. 模板

Flask 使用 Jinja 2 作为模板引擎。

标准上下文，默认在 Jinja2 模板中可用

- config

当前的配置对象 (flask.config)

- request

当前的请求对象 (flask.request)

- session

当前的会话对象 (flask.session)

- g

请求相关的全局变量 (flask.g)

- url_for()
- getflashedmessages()

7. 蓝图

可以定制templates, static目录

四. flask最佳实践

- env
- 代码结构
- 工厂函数
- config
- 请求钩子

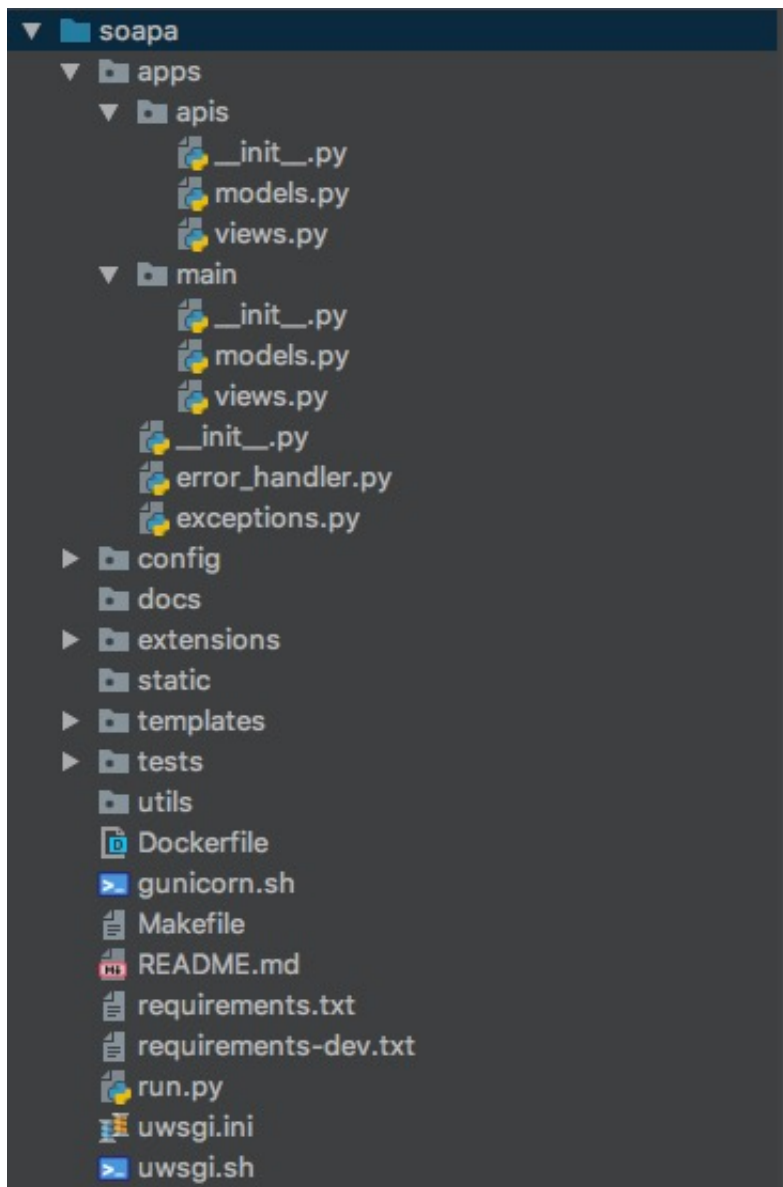
1. env

构建环境，避免多个项目环境之间依赖冲突

virtualenv

virtualenvwrapper

2. 代码结构



- 功能型

相同功能的模块放在一起

```
1 | yourapp/  
2 |     __init__.py  
3 |     static/  
4 |     templates/  
5 |         home/  
6 |         control_panel/  
7 |         admin/  
8 |     views/  
9 |         __init__.py  
10 |         home.py  
11 |         control_panel.py  
12 |         admin.py  
13 |     models.py
```

- 划分型

每个单独的功能或模块可以放在一个views文件或包里，公共服务接口可以放在上层位置，或作为单独的Blueprint (API 服务)或模块包。

```
1 | yourapp/  
2 |     __init__.py  
3 |     admin/  
4 |         __init__.py  
5 |         views.py  
6 |         static/  
7 |         templates/  
8 |     home/  
9 |         __init__.py  
10 |        views.py  
11 |        static/  
12 |        templates/  
13 |    control_panel/  
14 |        __init__.py  
15 |        views.py  
16 |        static/  
17 |        templates/  
18 |    models.py
```

可以混用

3. 工厂函数

Instead of creating your app as

```
1 | from flask import Flask  
2 |  
3 | app = Flask(__name__)
```

Python

wrap it in a function

```
1 def create_app(environment):
2     app = Application(environment)
3     if environment == "dev":
4         app.debug = True
5
6     #: 数据库
7
8     #: 注册蓝图
9     from apps.error_handler import error_bp
10    from apps.main.views import main_bp
11    from apps.apis.views import api_bp
12    blueprints = [api_bp, error_bp, main_bp]
13    app._init_blueprint(*blueprints)
14
15    #: 注册jinja上下文
16
17    return app
```

4.config

读取配置 `python current_app.config.get("key")`

- 避免配置写在多个地方
- 避免重构代码结构时引用异常

5.请求钩子

为了避免在每个视图函数中都使用重复的代码，Flask提供了注册通用函数的功能，注册的函数可在请求被分发到视图函数之前或之后调用。

- **before_first_request**

顾名思义，注册一个在处理第一个请求之前运行的函数

- **before_request**

注册一个在处理请求之前运行的函数。

- **after_request**

注册一个函数，如果没有未处理的异常抛出，在每次请求之后运行。

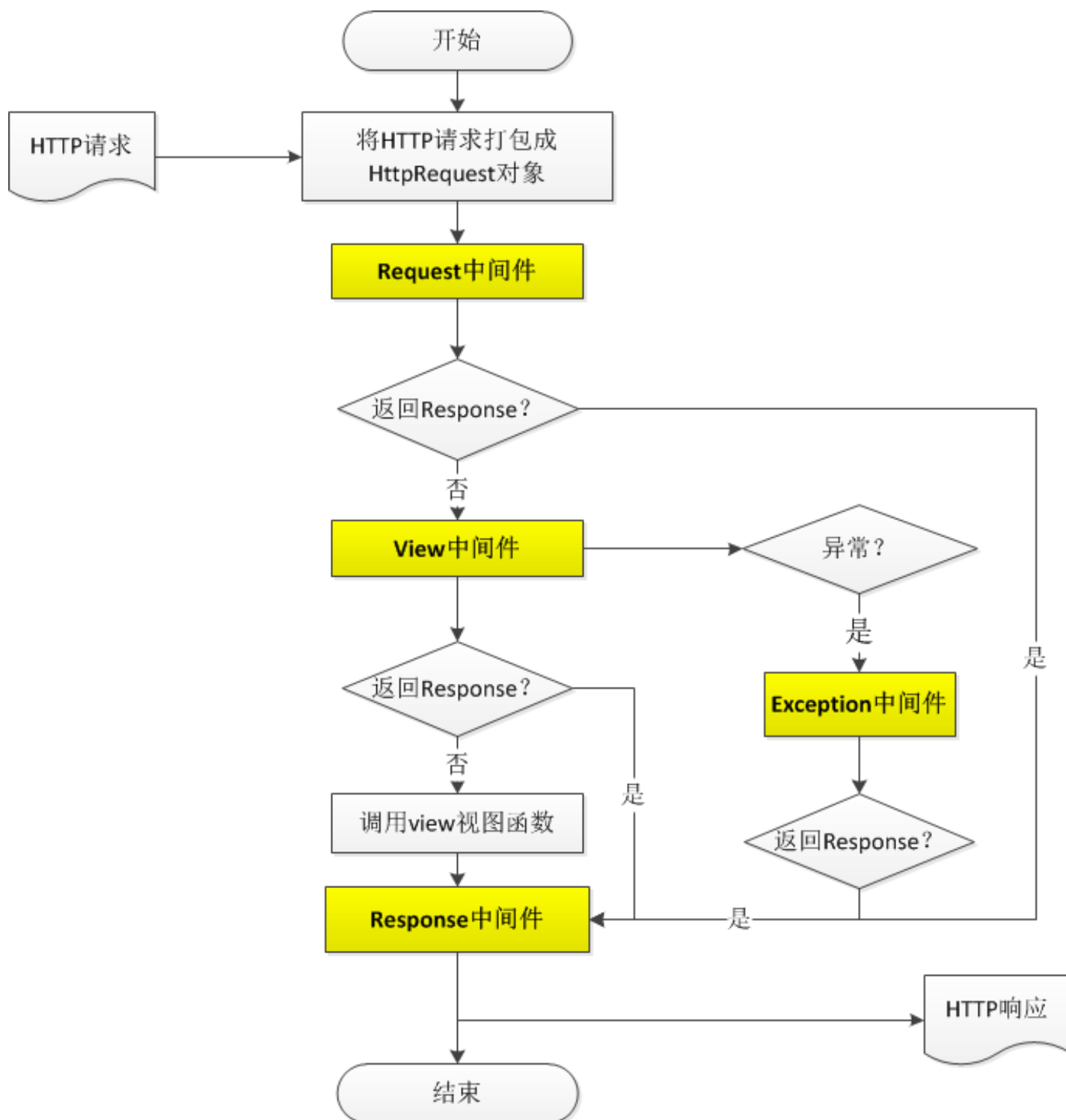
- **error_handler**

注册异常处理函数

- teardown_request

注册一个函数，即使有未处理的异常抛出，也在每次请求之后运行

类似于Django的中间件



6. flask.g

使用flask.g传递数据

- 在应用上下文之间销毁
- 线程安全

如果自己定义全局变量需要用线程锁锁住资源

参考：

[flask文档](#)

7.部署

1. nginx + gunicorn + gevent + flask
2. nginx + uwsgi + gevent + flask
3. 使用werkzeug开发

```
1 | app = create_app("dev")
2 |
3 | if __name__ == '__main__':
4 |     app.run()
5 |
```

Python

五. RPC

- Jsonrpc
- restful

远程过程调用

Jsonrpc

参考：

[json-rpc](#)

restful

参考：

[connexion](#)